

Introduction

Behavioral design patterns focus on how objects interact and communicate with each other, helping to define the flow of control in a system. These patterns make systems more flexible by allowing behavior to be selected or changed at runtime without altering the core logic.

Imagine a navigation app that can switch between driving, walking, or cycling routes. The algorithm used to calculate the path depends on the selected mode of travel. Instead of hardcoding all possible strategies inside one class, wouldn't it be better if each strategy was defined separately and chosen dynamically?

That's exactly what the **Strategy Pattern** enables. It allows a class to choose its behavior at runtime by encapsulating related algorithms into interchangeable objects. Let's explore the Strategy Pattern in detail in the upcoming sections.

Strategy Pattern

The **Strategy Pattern** is a behavioral design pattern that defines a family of algorithms, encapsulates each one into a separate class, and makes them interchangeable at runtime depending on the context.

Formal Definition

The Strategy Pattern is a behavioral design pattern that enables selecting an algorithm's behavior at runtime by defining a set of strategies (algorithms), each encapsulated in its own class, and making them interchangeable via a common interface.

It is primarily focused on changing the behavior of an object dynamically, without modifying its class. This promotes better organization of related algorithms and enhances code flexibility and scalability.

Real-Life Analogy

Consider how **Uber** matches a rider with a driver. The underlying algorithm may change depending on the context, like matching with the **nearest driver**, giving **priority to surge zones**, or choosing from an **airport queue**.

In this case:

- The **ride-matching service** is the **context**.
- The different matching algorithms (nearest, surge-priority, airport-queue) are the **strategies**.
- The **strategy interface** allows the system to switch between these algorithms seamlessly, depending on real-time conditions.

Similarly, in software, the Strategy Pattern allows a class to use different algorithms or behaviors at runtime, without altering its code structure, just like Uber switches matching strategies based on need.

Understanding the Problem

Let's say we are building a ride-matching service for a ride-hailing platform. The matching behavior changes depending on conditions such as proximity, surge areas, or airport queues.

Here's a naive implementation of this logic:

```
#include <iostream>
```

```

#include <string>

using namespace std;


// Class implementing Ride Matching Service
class RideMatchingService {

public:

    void matchRider(const string& riderLocation, const string&
matchingType) {

        // Match rider using different hardcoded strategies

        if (matchingType == "NEAREST") {

            // Find nearest driver

            cout << "Matching rider at " << riderLocation << " with nearest driver."
<< endl;

        } else if (matchingType == "SURGE_PRIORITY") {

            // Match based on surge area logic

            cout << "Matching rider at " << riderLocation << " based on surge
pricing priority." << endl;

        } else if (matchingType == "AIRPORT_QUEUE") {

            // Use FIFO-based airport queue logic

            cout << "Matching rider at " << riderLocation << " from airport queue."
<< endl;

        } else {

            cout << "Invalid matching strategy provided." << endl;

```

```

    }

}

};

// Client Code

int main() {

    RideMatchingService service;


    // Try different strategies

    service.matchRider("Downtown", "NEAREST");

    service.matchRider("City Center", "SURGE_PRIORITY");

    service.matchRider("Airport Terminal 1", "AIRPORT_QUEUE");


    return 0;

}


import java.util.*;


// Class implementing Ride Matching Service

class RideMatchingService {

    public void matchRider(String riderLocation, String matchingType) {

```

```

// Match rider using different hardcoded strategies

if (matchingType.equals("NEAREST")) {

    // Find nearest driver

    System.out.println("Matching rider at " + riderLocation + " with
nearest driver.");

} else if (matchingType.equals("SURGE_PRIORITY")) {

    // Match based on surge area logic

    System.out.println("Matching rider at " + riderLocation + " based on
surge pricing priority.");

} else if (matchingType.equals("AIRPORT_QUEUE")) {

    // Use FIFO-based airport queue logic

    System.out.println("Matching rider at " + riderLocation + " from
airport queue.");

} else {

    System.out.println("Invalid matching strategy provided.");

}

}

}

// Client Code

public class Main {

    public static void main(String[] args) {

```

```

RideMatchingService service = new RideMatchingService();

// Try different strategies

service.matchRider("Downtown", "NEAREST");

service.matchRider("City Center", "SURGE_PRIORITY");

service.matchRider("Airport Terminal 1", "AIRPORT_QUEUE");

}

}

# Class implementing Ride Matching Service

class RideMatchingService:

    def match_rider(self, rider_location, matching_type):

        # Match rider using different hardcoded strategies

        if matching_type == "NEAREST":

            # Find nearest driver

            print(f"Matching rider at {rider_location} with nearest driver.")

        elif matching_type == "SURGE_PRIORITY":

            # Match based on surge area logic

            print(f"Matching rider at {rider_location} based on surge pricing
priority.")

        elif matching_type == "AIRPORT_QUEUE":

            # Use FIFO-based airport queue logic

```

```

        print(f"Matching rider at {rider_location} from airport queue.")

    else:

        print("Invalid matching strategy provided.")


# Client Code

def main():

    service = RideMatchingService()


    # Try different strategies

    service.match_rider("Downtown", "NEAREST")

    service.match_rider("City Center", "SURGE_PRIORITY")

    service.match_rider("Airport Terminal 1", "AIRPORT_QUEUE")


if __name__ == "__main__":

    main()

```

Problems with This Approach:

Issue	Explanation
-------	-------------

Violation of Open/Closed Principle	Adding a new strategy (e.g., VIP rider matching) would require modifying the RideMatchingService class. This tightly couples strategy logic with the core class.
Code Becomes Messy	As more conditions are added, the number of if-else branches grows, making the code harder to maintain and read.
Difficult to Test or Reuse	Individual matching strategies are not reusable or testable in isolation. All logic is embedded inside a single method.
No Separation of Concerns	The class handles both coordination (service logic) and implementation (strategy logic), which reduces flexibility.

The Solution

The **Strategy Pattern** helps eliminate complex conditional logic by encapsulating each matching algorithm into its own class. The ride-matching service then delegates the decision-making to the selected strategy at runtime. This makes the system flexible, extensible, and easier to maintain.

Let's look at the implementation in code:

```
#include <iostream>
```



```

#include <string>

using namespace std;

// =====

// Strategy Interface

// =====

class MatchingStrategy {

public:

    virtual void match(const string& riderLocation) = 0;

    virtual ~MatchingStrategy() {}

};

// =====

// Concrete Strategy: Nearest Driver

// =====

class NearestDriverStrategy : public MatchingStrategy {

public:

    void match(const string& riderLocation) override {

        cout << "Matching with the nearest available driver to " <<
riderLocation << endl;

        // Distance-based matching logic

    }

}

```

```
};
```

```
// =====
```

```
// Concrete Strategy: Airport Queue
```

```
// =====
```

```
class AirportQueueStrategy : public MatchingStrategy {
```

```
public:
```

```
    void match(const string& riderLocation) override {
```

```
        cout << "Matching using FIFO airport queue for " << riderLocation <<  
endl;
```

```
        // Match first-in-line driver for airport pickup
```

```
    }
```

```
};
```

```
// =====
```

```
// Concrete Strategy: Surge Priority
```

```
// =====
```

```
class SurgePriorityStrategy : public MatchingStrategy {
```

```
public:
```

```
    void match(const string& riderLocation) override {
```

```
        cout << "Matching rider using surge pricing priority near " <<  
riderLocation << endl;
```

```

        // Prioritize high-surge zones or premium drivers
    }

};

// =====

// Context Class: RideMatchingService

// =====

class RideMatchingService {

private:

    MatchingStrategy* strategy;

public:

    // Constructor injection of strategy

    RideMatchingService(MatchingStrategy* strategy) {

        this->strategy = strategy;

    }

    // Setter injection for changing strategy dynamically

    void setStrategy(MatchingStrategy* strategy) {

        this->strategy = strategy;

    }

```

```

// Delegates the matching logic to the strategy
void matchRider(const string& location) {

    strategy->match(location);

}

};

// =====

// Client Code

// =====

int main() {

    // Using airport queue strategy

    RideMatchingService rideMatchingService(new
AirportQueueStrategy());

    rideMatchingService.matchRider("Terminal 1");


    // Using nearest driver strategy and later switching to surge priority

    RideMatchingService rideMatchingService2(new
NearestDriverStrategy());

    rideMatchingService2.matchRider("Downtown");

    rideMatchingService2.setStrategy(new SurgePriorityStrategy());

    rideMatchingService2.matchRider("Downtown");

```

```

        return 0;
    }

import java.util.*;

// =====

// Strategy Interface

// =====

interface MatchingStrategy {

    void match(String riderLocation);

}

// =====

// Concrete Strategy: Nearest Driver

// =====

class NearestDriverStrategy implements MatchingStrategy {

    @Override

    public void match(String riderLocation) {

        System.out.println("Matching with the nearest available driver to " +
riderLocation);

        // Distance-based matching logic

```

```

    }
}

// =====

// Concrete Strategy: Airport Queue

// =====

class AirportQueueStrategy implements MatchingStrategy {

    @Override

    public void match(String riderLocation) {

        System.out.println("Matching using FIFO airport queue for " +
riderLocation);

        // Match first-in-line driver for airport pickup

    }

}

// =====

// Concrete Strategy: Surge Priority

// =====

class SurgePriorityStrategy implements MatchingStrategy {

    @Override

    public void match(String riderLocation) {

```

```
        System.out.println("Matching rider using surge pricing priority near " +  
riderLocation);
```

```
        // Prioritize high-surge zones or premium drivers  
    }  
}
```

```
// =====
```

```
// Context Class: RideMatchingService
```

```
// =====
```

```
class RideMatchingService {
```

```
    private MatchingStrategy strategy;
```

```
    // Constructor injection of strategy
```

```
    public RideMatchingService(MatchingStrategy strategy) {
```

```
        this.strategy = strategy;
```

```
    }
```

```
    // Setter injection for changing strategy dynamically
```

```
    public void setStrategy(MatchingStrategy strategy) {
```

```
        this.strategy = strategy;
```

```
    }
```

```

// Delegates the matching logic to the strategy

public void matchRider(String location) {

    strategy.match(location);

}

}

// =====

// Client Code

// =====

public class Main {

    public static void main(String[] args) {

        // Using airport queue strategy

        RideMatchingService rideMatchingService = new
RideMatchingService(new AirportQueueStrategy());

        rideMatchingService.matchRider("Terminal 1");

        // Using nearest driver strategy and later switching to surge priority

        RideMatchingService rideMatchingService2 = new
RideMatchingService(new NearestDriverStrategy());

        rideMatchingService2.matchRider("Downtown");

        rideMatchingService2.setStrategy(new SurgePriorityStrategy());

        rideMatchingService2.matchRider("Downtown");

```



```
}  
}
```

```
from abc import ABC, abstractmethod
```

```
# =====
```

```
# Strategy Interface
```

```
# =====
```

```
class MatchingStrategy(ABC):
```

```
    @abstractmethod
```

```
    def match(self, rider_location):
```

```
        pass
```

```
# =====
```

```
# Concrete Strategy: Nearest Driver
```

```
# =====
```

```
class NearestDriverStrategy(MatchingStrategy):
```

```
    def match(self, rider_location):
```

```
        print(f"Matching with the nearest available driver to {rider_location}")
```

```
        # Distance-based matching logic
```

```

# =====

# Concrete Strategy: Airport Queue

# =====

class AirportQueueStrategy(MatchingStrategy):

    def match(self, rider_location):

        print(f"Matching using FIFO airport queue for {rider_location}")

        # Match first-in-line driver for airport pickup


# =====

# Concrete Strategy: Surge Priority

# =====

class SurgePriorityStrategy(MatchingStrategy):

    def match(self, rider_location):

        print(f"Matching rider using surge pricing priority near
{rider_location}")

        # Prioritize high-surge zones or premium drivers


# =====

# Context Class: RideMatchingService

# =====

class RideMatchingService:

    def __init__(self, strategy):

```

```

        self.strategy = strategy # Constructor injection of strategy

    def set_strategy(self, strategy):

        self.strategy = strategy # Setter injection to change strategy
        dynamically

    def match_rider(self, location):

        self.strategy.match(location) # Delegates the matching logic to the
        strategy

# =====

# Client Code

# =====

def main():

    # Using airport queue strategy

    ride_matching_service =
    RideMatchingService(AirportQueueStrategy())

    ride_matching_service.match_rider("Terminal 1")


    # Using nearest driver strategy and later switching to surge priority

    ride_matching_service2 =
    RideMatchingService(NearestDriverStrategy())

    ride_matching_service2.match_rider("Downtown")

```

```
ride_matching_service2.set_strategy(SurgePriorityStrategy())
```

```
ride_matching_service2.match_rider("Downtown")
```

```
if __name__ == "__main__":
```

```
    main()
```

How This Solves the Earlier Problems

Problem in Old Approach	How Strategy Pattern Solves It
Violation of Open/Closed Principle	New strategies can be added without modifying existing service code, just create a new class implementing MatchingStrategy .
Code Becomes Messy	Eliminates complex if-else logic by delegating behavior to separate classes.
Difficult to Test or Reuse	Each strategy is independently testable and reusable across services or contexts.
No Separation of Concerns	RideMatchingService is only concerned with coordination, actual logic lies in strategy classes.

Suitable Scenarios for Strategy Pattern

The Strategy Pattern is an ideal choice in the following scenarios:

- **Multiple Interchangeable Algorithms:**
When a system supports different algorithms or behaviors that can be swapped in and out based on context or configuration.
 - **Compliance with Open/Closed Principle (OCP):**
When new strategies need to be introduced without modifying the existing business logic, keeping the core code closed for modification and open for extension.
 - **Elimination of Conditionals:**
When large blocks of **if-else** or **switch** statements are used to select behavior, Strategy Pattern helps to cleanly separate these into dedicated classes.
 - **Behavior-Specific Unit Testing**
When there's a need to test behaviors independently and isolate them from the context, Strategy Pattern offers clear test boundaries.
 - **Runtime Behavior Selection**
When the behavior of a class needs to be selected dynamically during execution based on user input, configuration, or environment.
-

Pros and Cons

Pros

- **Supports the Open/Closed Principle (OCP):**
New strategies can be added without modifying existing code, keeping the system extensible.
- **Easy to Add New Behaviors:**
Each behavior is encapsulated in its own class, making it simple to plug in new logic.
- **Enables Runtime Behavior Changes:**
Behavior can be changed dynamically at runtime by swapping strategy objects.

- **Encourages Composition Over Inheritance:**

Promotes flexible design by favoring object composition rather than rigid class hierarchies.

Cons

- **May Lead to Too Many Small Classes:**

Each strategy is implemented in a separate class, which can increase code volume.

- **Requires Awareness of All Strategies:**

The client needs to know which strategies exist and when to use each one.

- **Slight Overhead Due to Interfaces:**

Involves extra structure around interfaces, which may be unnecessary for simple logic.

- **Slightly More Complex Than `if-else`:**

For very simple cases, the Strategy Pattern may introduce more complexity than needed.

Class Diagram

